

SISTEMAS OPERACIONAIS

Processos e Threads

Uma síntese do Capítulo 2 de "Sistemas Operacionais Modernos", de Andrew S. Tanenbaum e Herbert Bos: processos, threads, sincronização e escalonamento.



Agenda



01 O conceito de processo

02 Criação, término e hierarquias

03 Estados de um processo

04 Threads: motivação e implementação

05 Condições de corrida e exclusão mútua

06 Primitivas de sincronização e problemas clássicos

07 Escalonamento: objetivos e algoritmos

08 Tempo real, threads e síntese final



O Conceito de Processo

Processo: uma instância de um programa em execução, com seus próprios valores de contador de programa, registradores e variáveis.

Programa (*entidade passiva, uma "receita"*) é diferente de **processo** (*entidade ativa, o "cozinheiro" executando a receita*).

O modelo de processos organiza todo o software executável — muitas vezes incluindo o próprio SO — em processos sequenciais, cada um com a ilusão de possuir sua própria CPU dedicada.

CPU Virtual e Pseudoparalelismo

Em vez de gerenciar diretamente a disputa de vários programas físicos pela CPU, o SO cria a ilusão de que cada processo tem sua própria CPU exclusiva — ainda que a CPU real alterne rapidamente entre eles.

Essa alternância rápida é o pseudoparalelismo: múltiplos processos parecem executar ao mesmo tempo em uma única CPU.

Criação, Término e Hierarquias



Criação de Processos

- Inicialização do sistema
- Chamada de sistema de criação por um processo em execução (fork)
- Requisição do usuário para criar um novo processo
- Início de uma tarefa em lote (batch)

Término de Processos

- Saída voluntária (término normal)
- Saída por erro voluntária (o processo detecta um erro fatal)
- Erro fatal involuntário (bug: instrução ilegal, divisão por zero)
- Eliminação por outro processo (via chamada de sistema)

Hierarquias: no UNIX, um processo e seus descendentes formam uma árvore com o "init" como ancestral comum. O Windows não tem esse conceito — todo processo criado recebe um "handle" que permite ao pai controlá-lo, mas não há hierarquia formal.

Estados de um Processo



Implementação: o SO mantém uma tabela de processos, com um bloco de controle de processo (PCB) por entrada, guardando registradores, contador de programa, ponteiro de pilha e estado — permitindo retomar cada processo exatamente de onde parou.



Por que Usar Threads?

Threads (processos leves): compartilham o espaço de endereçamento do processo (código, dados, arquivos abertos), mas cada uma mantém seu próprio contador de programa, registradores e pilha.

1

Paralelismo na aplicação

Um servidor pode atender várias requisições simultaneamente dentro do mesmo processo.

2

Mais leves que processos

Criar e destruir uma thread é mais barato — não exige duplicar o espaço de endereçamento.

3

Sobrepõem CPU e E/S

Uma thread continua processando enquanto outra aguarda uma operação de E/S.

4

Exploram multiprocessadores

Múltiplas threads podem executar em paralelo real, em núcleos distintos.



Implementação de Threads

Abordagem	Como funciona	Vantagens / Desvantagens
Espaço do usuário	Biblioteca de runtime gerencia as threads; o núcleo não sabe que existem	Troca rapidíssima entre threads; porém uma chamada bloqueante trava todo o processo
Núcleo (kernel)	O próprio SO cria, escalona e gerencia as threads	Uma thread bloqueada não trava as demais; porém cada operação exige chamada de sistema (mais custosa)
Híbrida (many-to-many)	Várias threads de usuário mapeadas a menos threads de núcleo	Busca o melhor dos dois mundos, à custa de maior complexidade

Ativações do escalonador (scheduler activations): técnica que notifica o espaço do usuário quando uma thread bloqueia, buscando unir a eficiência das threads de usuário à correção das threads de núcleo. **Threads pop-up:** criadas automaticamente para tratar mensagens recebidas, evitando manter threads ociosas à espera.



Condições de Corrida e Região Crítica

Condição de Corrida (Race Condition)

Situação em que o resultado final depende da ordem exata em que operações concorrentes são executadas — como no exemplo clássico do spooler de impressão, quando dois processos podem escrever no mesmo slot da fila.

Região crítica: trecho de código que acessa memória ou recursos compartilhados e deve ser protegido contra acesso concorrente.

4 Condições para uma Boa Solução

- Dois processos jamais podem estar simultaneamente em suas regiões críticas
- Nenhuma suposição pode ser feita sobre velocidades ou número de CPUs
- Nenhum processo fora de sua região crítica pode bloquear outros
- Nenhum processo deve esperar eternamente para entrar em sua região crítica

Exclusão Mútua com Espera Ocupada



Técnica	Ideia	Limitação
Desabilitar interrupções	Impede a troca de contexto durante a região crítica	Perigoso em mãos de processos de usuário; ineficaz em multiprocessadores
Variáveis de bloqueio (lock)	Uma variável compartilhada sinaliza recurso ocupado	A checagem e o ajuste da variável não são atômicos — ainda sujeito a condição de corrida
Alternância estrita	Processos se revezam rigidamente por turnos	Um processo lento força o outro a esperar sem necessidade
Solução de Peterson	Combina variável de "vez" com vetor de intenção	Correta e elegante (software puro), mas ainda usa espera ocupada
TSL / XCHG	Instruções atômicas de hardware para testar-e-ajustar	Base real da exclusão mútua, mas continua gastando CPU em espera

Primitivas de Sincronização de Alto Nível



Primitiva	Ideia central
Dormir e acordar	Chamadas de sistema (sleep / wakeup) que bloqueiam um processo até ser explicitamente acordado — base do produtor-consumidor, mas sujeita a sinais perdidos
Semáforos (Dijkstra)	Variável inteira com operações atômicas down/up — usados para exclusão mútua e para sincronizar a ordem de eventos
Mutexes	Versão simplificada do semáforo (livre/ocupado), voltada à exclusão mútua entre threads
Monitores	Módulo de linguagem que agrupa dados e procedimentos, com exclusão mútua garantida automaticamente pelo compilador
Troca de mensagens	Comunicação via send/receive — essencial em sistemas distribuídos sem memória compartilhada
Barreiras	Nenhum processo do grupo avança até que todos alcancem a barreira — útil em computação paralela em fases

Problemas Clássicos de Sincronização



Produtor-Consumidor

Um processo produz itens em um buffer de tamanho fixo; outro os consome. Exige sincronização para o produtor não escrever em buffer cheio nem o consumidor ler de buffer vazio.

Jantar dos Filósofos

Cinco filósofos alternam entre pensar e comer, disputando garfos compartilhados — ilustra deadlock e inanição (starvation) na disputa por múltiplos recursos.

Leitores e Escritores

Modela o acesso a uma base de dados compartilhada: permite múltiplos leitores simultâneos, mas exige acesso exclusivo para um escritor.



Escalonamento: Quando e Para Quê

Quando escalonar: na criação ou término de um processo, quando um processo bloqueia, ou em uma interrupção de E/S. Algoritmos **não preemptivos** deixam o processo rodar até bloquear ou terminar; algoritmos **preemptivos** podem suspendê-lo (tipicamente ao fim de um quantum).

Todos os Sistemas

Justiça, eficácia da política e balanceamento entre CPU e E/S.

Sistemas em Lote

Maximizar throughput e utilização da CPU; minimizar o turnaround time.

Sistemas Interativos

Minimizar o tempo de resposta e atender à proporcionalidade esperada pelo usuário.

Tempo Real

Cumprir prazos (deadlines) com previsibilidade, evitando perda de dados.



Escalonamento em Sistemas em Lote

Algoritmo	Lógica
FCFS (First-Come, First-Served)	Executa os processos na ordem de chegada, sem preempção — simples, mas processos curtos podem esperar atrás de processos longos
SJF (Shortest Job First)	Escolhe, entre os disponíveis, o processo com menor tempo de execução estimado — ótimo para minimizar a espera média quando todos chegam juntos
SRTN (Shortest Remaining Time Next)	Versão preemptiva do SJF: um processo novo com tempo menor que o restante do atual assume a CPU imediatamente

Ambiente típico: processamento sem interação do usuário, em que o foco é a vazão e a utilização eficiente da CPU ao longo do tempo.

Escalonamento em Sistemas Interativos



Algoritmo	Lógica
Round-Robin	Cada processo recebe um quantum; ao esgotá-lo, volta ao fim da fila de prontos
Prioridades	Processo de maior prioridade executa primeiro; prioridades podem ser ajustadas para evitar inanição
Múltiplas filas	Classes de prioridade, cada uma com seu próprio quantum (crescente para prioridades menores)
Mais curto processo em seguida	Aplica a ideia do SJF ao ambiente interativo, estimando o tempo com base no histórico

Algoritmo	Lógica
Escalonamento garantido	Promete a cada um dos n processos $\sim 1/n$ da CPU, comparando uso real ao prometido
Loteria (lottery scheduling)	Distribui "bilhetes" de CPU; um bilhete sorteado a cada decisão — proporcional e probabilístico
Justo (fair-share)	Considera o usuário dono do processo, evitando que um usuário monopolize a CPU

Tempo Real e Escalonamento de Threads



Sistemas de Tempo Real

Tarefas rígidas (hard): um prazo perdido é uma falha do sistema.

Tarefas flexíveis (soft): um prazo perdido é indesejável, mas tolerável.

O algoritmo de escalonamento deve ser previsível, priorizando o cumprimento de prazos sobre métricas médias de desempenho.

Escalonamento de Threads

Depende da implementação: threads de núcleo são escalonadas individualmente pelo SO (que desconhece o processo "dono"), enquanto threads de usuário são escalonadas por um executor de runtime dentro do próprio processo — que pode até usar uma política totalmente diferente da do SO.



Síntese

O capítulo constrói progressivamente a base da concorrência nos sistemas operacionais modernos, equilibrando quatro camadas:



Processo

A abstração de CPU virtual que organiza o software executável



Thread

Paralelismo dentro do mesmo espaço de endereçamento



Sincronização

Cooperação segura entre entidades concorrentes



Escalonamento

A decisão de quem, entre os prontos, recebe a CPU



Referências Bibliográficas

1

TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Capítulo 2 – Processos e Threads. ISBN 978-85-430-0567-6.

2

TANENBAUM, Andrew S.; BOS, Herbert. Modern Operating Systems. 4th ed. Boston: Pearson, 2014. Chapter 2 – Processes and Threads (edição original em inglês, usada para conferência de termos técnicos).

3

Wikipedia. Modern Operating Systems — histórico de edições da obra. Disponível em: en.wikipedia.org/wiki/Modern_Operating_Systems.

Material de apoio para o projeto "Sistemas Operacionais" — resumos, questionários e slides.